LeetCode Comprehensive Revision Cheatsheet

Contents

1 Pattern Classification **2**
- 1.1 1. Array Manipulation . . . 2
- 1.2 2. HashMap/Frequency Counting . . . 2
- 1.3 3. Stack . . . 2
- 1.4 4. Sliding Window . . . 2
- 1.5 5. Two Pointers . . . 2
- 1.6 6. Binary Search . . . 2
- 1.7 7. Dynamic Programming (Kadane's) . . . 2

2 Core Fundamentals **3**
- 2.1 1. Sliding Window - When & How . . . 3
- 2.2 2. Two Pointers - Key Decisions . . . 3
- 2.3 3. Binary Search - Loop Conditions . . . 4
- 2.4 4. HashMap Optimization . . . 5

3 Problem-by-Problem Guide **6**
- 3.1 1. Product of Array Except Self . . . 6
- 3.2 2. Subarray Sum Equals K . . . 6
- 3.3 3. Top K Frequent Elements . . . 7
- 3.4 4. Valid Parentheses . . . 8
- 3.5 5. Group Anagrams . . . 9
- 3.6 6. Longest Substring Without Repeating Characters . . . 9
- 3.7 7. Minimum Window Substring . . . 10
- 3.8 8. Longest Repeating Character Replacement . . . 11
- 3.9 9. Maximum Average Subarray . . . 12
- 3.10 10. Container With Most Water . . . 13
- 3.11 11. Trapping Rain Water . . . 13
- 3.12 12. Find Minimum in Rotated Sorted Array . . . 14
- 3.13 13. Search in Rotated Sorted Array . . . 15
- 3.14 14. Maximum Subarray (Kadane's Algorithm) . . . 16

4 Skeleton Code Templates **17**
- 4.1 Template 1: Variable Sliding Window . . . 17
- 4.2 Template 2: Fixed Sliding Window . . . 17
- 4.3 Template 3: Two Pointers (Opposite Ends) . . . 17
- 4.4 Template 4: Binary Search (Find Position) . . . 18
- 4.5 Template 5: Binary Search (Find Value) . . . 18
- 4.6 Template 6: HashMap Frequency Counting . . . 18

5 Common Gotchas & Pitfalls **20**
5.1 1. Integer Comparison in HashMap . . . 20
5.2 2. Sliding Window - When to Update Result . . . 20
5.3 3. Array Bounds with Pointers . . . 20
5.4 4. Binary Search - Infinite Loop . . . 20
5.5 5. Substring End Index . . . 21
5.6 6. Early Return in Loops . . . 21
5.7 7. Stack Empty Check . . . 21
5.8 8. Max/Min Initialization . . . 21

6 Your Knowledge Gaps & Areas to Review **23**
6.1 1. Confusing Similar Problems . . . 23
6.2 2. Order of Operations in Same Iteration . . . 23
6.3 3. Binary Search Loop Conditions . . . 23
6.4 4. Math.max() in Sliding Window . . . 24
6.5 5. When HashMap vs Array . . . 24

7 Quick Reference Tables **25**
7.1 Complexity Cheat Sheet . . . 25
7.2 When to Use Which Data Structure . . . 25
7.3 Optimization Checklist . . . 25

8 Tips & Tricks **26**
8.1 1. Recognize the Pattern . . . 26
8.2 2. Ask These Questions . . . 26
8.3 3. Debugging Checklist . . . 26
8.4 4. Optimization Strategies . . . 27
8.5 5. Edge Cases to Always Test . . . 27

9 Mental Models **28**
9.1 Sliding Window Mental Model . . . 28
9.2 Two Pointers Mental Model . . . 28
9.3 Binary Search Mental Model . . . 28
9.4 Stack Mental Model . . . 28

10 Study Plan **29**
10.1 Week 1: Master the Patterns . . . 29
10.2 Week 2: Solidify with Variations . . . 29
10.3 Daily Practice Routine . . . 29

11 Before Your Interview Checklist **30**

1 Pattern Classification

1.1 1. Array Manipulation

- Product of Array Except Self
- Subarray Sum Equals K

1.2 2. HashMap/Frequency Counting

- Top K Frequent Elements
- Group Anagrams
- Subarray Sum Equals K

1.3 3. Stack

- Valid Parentheses

1.4 4. Sliding Window

- Longest Substring Without Repeating Characters
- Minimum Window Substring
- Longest Repeating Character Replacement
- Maximum Average Subarray

1.5 5. Two Pointers

- Container With Most Water
- Trapping Rain Water

1.6 6. Binary Search

- Find Minimum in Rotated Sorted Array
- Search in Rotated Sorted Array

1.7 7. Dynamic Programming (Kadane's)

- Maximum Subarray

2 Core Fundamentals

2.1 1. Sliding Window - When & How

SLIDING WINDOW PATTERN

Use When:

- Contiguous subarray/substring problem
- Need to find optimal window (min/max length)
- Can expand/shrink window based on conditions

Two Types:
Fixed Size Window:

- Window size is constant (k)
- Slide by adding right, removing left
- Example: Maximum Average Subarray

Variable Size Window:

- Window grows and shrinks based on condition
- Expand: Add right element
- Shrink: Remove left until valid
- Examples: Minimum Window, Longest Substring

Template:

```
left = 0
for right in range(n):
    # Add right element to window
    window.add(arr[right])

    # Shrink if condition violated
    while (condition_violated):
        window.remove(arr[left])
        left++

    # Update result
    result = update(result, window)
```

2.2 2. Two Pointers - Key Decisions

TWO POINTER PATTERN

Critical Question: Which pointer to move?
Strategy 1: Move both based on comparison

- Compare elements at l and r
- Move based on which needs to change

- Example: Container With Most Water

Strategy 2: Move based on running max/min

- Track leftMax and rightMax
- Move pointer with smaller max
- Example: Trapping Rain Water

Template:

```
left = 0, right = n - 1

while left < right:
    # Calculate current result

    # Decide which pointer to move
    if (condition):
        left++
    else:
        right--
```

2.3 3. Binary Search - Loop Conditions

BINARY SEARCH - CRITICAL CHOICE

left < right:

- Finding a POSITION (minimum, boundary, peak)
- Answer guaranteed to exist
- Use: right = mid (keep mid in search space)
- Exit: left == right (converged)
- Return: nums[left]

left <= right:

- Finding EXACT VALUE (target search)
- Answer may not exist
- Use: right = mid - 1, left = mid + 1
- Exit: left > right (exhausted)
- Return: index or -1

Rule of Thumb:
If you see "right = mid" → use "left < right"
If you see "right = mid - 1" → use "left <= right"

2.4 4. HashMap Optimization

HASHMAP to ARRAY

When to optimize:

- Character problems (ASCII/lowercase letters)
- Limited value range
- Need O(1) space instead of O(k)

HashMap: Map<Character, Integer>

- Space: O(k) where k = unique elements
- Flexible: any character set
- Slower: hash operations

Array: int[26] or int[128]

- Space: O(1) constant
- Faster: direct indexing
- Limited: ASCII only

Conversion:

```
map.get(c) -> arr[c - 'a']  // lowercase
map.get(c) -> arr[c - 'A']  // uppercase
map.get(c) -> arr[c]        // ASCII
```

3 Problem-by-Problem Guide

3.1 1. Product of Array Except Self

Problem: Return array where output[i] = product of all elements except nums[i]
Key Insight: Use prefix and suffix products
Complexity:

- Time: O(n)
- Space: O(1) - output array doesn't count

Skeleton Code:

```java
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];

    // Build prefix products
    result[0] = 1;
    for (int i = 1; i < n; i++) {
        result[i] = result[i-1] * nums[i-1];
    }

    // Multiply by suffix products
    int suffix = 1;
    for (int i = n-1; i >= 0; i--) {
        result[i] *= suffix;
        suffix *= nums[i];
    }

    return result;
}
```

Gotchas:

X Can't use division (what if zeros?)

X Don't create separate prefix/suffix arrays (space!)

OK Build prefix in result array, then multiply suffix in-place

3.2 2. Subarray Sum Equals K

Problem: Count subarrays with sum = k
Key Insight: Use prefix sum + HashMap to track sum frequencies
Complexity:

- Time: O(n)
- Space: O(n) - HashMap

Skeleton Code:

```java
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1);  // Base case

    int sum = 0, count = 0;

    for (int num : nums) {
        sum += num;
```

```
9
10          // Check if (sum - k) exists
11          if (prefixCount.containsKey(sum - k)) {
12              count += prefixCount.get(sum - k);
13          }
14
15          // Add current sum to map
16          prefixCount.put(sum, prefixCount.getOrDefault(sum, 0) + 1);
17      }
18
19      return count;
20  }
```

Gotchas:

X Don't forget base case: map.put(0, 1)

X Can't use sliding window (negative numbers!)

OK Formula: if prefixSum[j] - prefixSum[i] = k, then prefixSum[i] = prefixSum[j] - k

Why HashMap?
Need to find: prefixSum[i] such that prefixSum[j] - prefixSum[i] = k
Rearrange: prefixSum[i] = prefixSum[j] - k
HashMap lets us check if (currentSum - k) was seen before

3.3 3. Top K Frequent Elements

Problem: Find k most frequent elements
Key Insight: Bucket sort by frequency
Complexity:

- Time: O(n)
- Space: O(n)

Skeleton Code:

```
1  public int[] topKFrequent(int[] nums, int k) {
2      // Count frequencies
3      Map<Integer, Integer> count = new HashMap<>();
4      for (int num : nums) {
5          count.put(num, count.getOrDefault(num, 0) + 1);
6      }
7
8      // Bucket sort
9      List<Integer>[] buckets = new List[nums.length + 1];
10     for (int num : count.keySet()) {
11         int freq = count.get(num);
12         if (buckets[freq] == null) {
13             buckets[freq] = new ArrayList<>();
14         }
15         buckets[freq].add(num);
16     }
17
18     // Collect top k
19     int[] result = new int[k];
20     int idx = 0;
21     for (int i = buckets.length - 1; i >= 0 && idx < k; i--) {
22         if (buckets[i] != null) {
23             for (int num : buckets[i]) {
```

```
                    result[idx++] = num;
                    if (idx == k) return result;
                }
            }
        }

        return result;
    }
```

Gotchas:

X Don't use PriorityQueue (O(n log k) - slower)

OK Bucket sort is O(n) - optimal

! Buckets are Array of ArrayLists, NOT LinkedList!

! Frequency range: [1, n], so need n+1 buckets

3.4 4. Valid Parentheses

Problem: Check if brackets are balanced
Key Insight: Use stack to match closing with opening
Complexity:

- Time: O(n)
- Space: O(n)

Skeleton Code:

```
public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    Map<Character, Character> pairs = Map.of(
        ')', '(',
        '}', '{',
        ']', '['
    );

    for (char c : s.toCharArray()) {
        if (!pairs.containsKey(c)) {
            // Opening bracket
            stack.push(c);
        } else {
            // Closing bracket
            if (stack.isEmpty() || stack.pop() != pairs.get(c)) {
                return false;
            }
        }
    }

    return stack.isEmpty();
}
```

Gotchas:

X Don't forget final stack.isEmpty() check

X Check stack.isEmpty() BEFORE pop()

OK Map closing to opening for cleaner code

3.5 5. Group Anagrams

Problem: Group strings that are anagrams
Key Insight: Use sorted string or character count as key
Complexity:

- Sorting: O(n × k log k) where k = max string length
- Counting: O(n × k)
- Space: O(n × k)

Skeleton Code (Counting - Faster):

```java
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> groups = new HashMap<>();

    for (String s : strs) {
        // Create frequency key
        int[] count = new int[26];
        for (char c : s.toCharArray()) {
            count[c - 'a']++;
        }
        String key = Arrays.toString(count);

        // Add to group
        groups.putIfAbsent(key, new ArrayList<>());
        groups.get(key).add(s);
    }

    return new ArrayList<>(groups.values());
}
```

Gotchas:

! Arrays.toString(count) is simplest for key

! Alternative: Build string "1#0#2#..." for counts

OK Counting is faster than sorting for longer strings

3.6 6. Longest Substring Without Repeating Characters

Problem: Find longest substring with all unique characters
Key Insight: Sliding window + HashMap to track last seen index
Complexity:

- Time: O(n)
- Space: O(min(n, m)) where m = charset size

Skeleton Code:

```java
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int left = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);

        // If duplicate, move left past previous occurrence
        if (lastSeen.containsKey(c)) {
            left = Math.max(left, lastSeen.get(c) + 1);
```

```
            }

            lastSeen.put(c, right);
            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }
```

Gotchas:

! MUST use Math.max(left, lastSeen.get(c) + 1)

X Don't just do: left = lastSeen.get(c) + 1

- **Why?** Previous character might be before current left!

Example:

```
s = "abba"
When right=3 (second 'a'):
lastSeen['a'] = 0
But left is already at 2 (after first 'b')
If we do left = 0 + 1 = 1, we go BACKWARDS! [X]
Must do: left = Math.max(2, 1) = 2 [OK]
```

3.7 7. Minimum Window Substring

Problem: Find smallest window in s containing all characters of t
Key Insight: Expand window until valid, then shrink to minimize
Complexity:

- Time: O(n + m)
- Space: O(k) or O(1) with array

Skeleton Code:

```
public String minWindow(String s, String t) {
    if (t.isEmpty()) return "";

    // Count target characters
    Map<Character, Integer> countT = new HashMap<>();
    for (char c : t.toCharArray()) {
        countT.put(c, countT.getOrDefault(c, 0) + 1);
    }

    Map<Character, Integer> window = new HashMap<>();
    int have = 0, need = countT.size();
    int[] res = {-1, -1};
    int resLen = Integer.MAX_VALUE;
    int l = 0;

    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        window.put(c, window.getOrDefault(c, 0) + 1);

        // Check if this character requirement met
        if (countT.containsKey(c) &&
            window.get(c).equals(countT.get(c))) {
            have++;
```

```
        }

        // Shrink window while valid
        while (have == need) {
            // Update result
            if ((r - l + 1) < resLen) {
                resLen = r - l + 1;
                res[0] = l;
                res[1] = r;
            }

            // Remove from left
            char leftChar = s.charAt(l);
            window.put(leftChar, window.get(leftChar) - 1);
            if (countT.containsKey(leftChar) &&
                window.get(leftChar) < countT.get(leftChar)) {
                have--;
            }
            l++;
        }
    }

    return resLen == Integer.MAX_VALUE ? "" :
           s.substring(res[0], res[1] + 1);
}
```

Gotchas:

! Track unique characters matched, not total count

! Use .equals() for Integer comparison, not ==

! need = countT.size() (unique chars), not sum of counts

OK Check exact match: window.get(c).equals(countT.get(c))

3.8 8. Longest Repeating Character Replacement

Problem: Longest substring after replacing $\leq$ k characters
Key Insight: Window valid if (windowSize - maxFreq) $\leq$ k
Complexity:

- Time: O(n)
- Space: O(1) - fixed array size 26

Skeleton Code:

```
public int characterReplacement(String s, int k) {
    int[] freq = new int[26];
    int left = 0, maxCount = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        freq[s.charAt(right) - 'A']++;
        maxCount = Math.max(maxCount, freq[s.charAt(right) - 'A']);

        // Shrink if too many replacements needed
        while ((right - left + 1) - maxCount > k) {
            freq[s.charAt(left) - 'A']--;
            left++;
        }

```

```
            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }
```

Gotchas:

! maxCount NEVER decreases (optimization!)

! Formula: replacements = windowSize - mostFrequentChar

OK Don't recalculate maxCount on shrink (not needed!)

Why maxCount doesn't decrease?
We only care about finding BETTER windows. If maxCount was 5 before, we've seen a window with 5 of same char. We only want windows with MORE than 5 (to get longer length). Keeping maxCount high makes condition stricter = only find better windows.

3.9 9. Maximum Average Subarray

Problem: Maximum average of k consecutive elements
Key Insight: Fixed-size sliding window
Complexity:

- Time: O(n)
- Space: O(1)

Skeleton Code:

```
public double findMaxAverage(int[] nums, int k) {
    // Calculate first window
    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += nums[i];
    }

    int maxSum = sum;

    // Slide window
    for (int i = k; i < nums.length; i++) {
        sum = sum - nums[i - k] + nums[i];
        maxSum = Math.max(maxSum, sum);
    }

    return (double) maxSum / k;
}
```

Gotchas:

X Don't recalculate sum for each window (O(n×k))

OK Slide: remove left, add right (O(n))

! Return double, not int

3.10 10. Container With Most Water

Problem: Maximum area between two lines
Key Insight: Move pointer of shorter height
Complexity:

- Time: O(n)
- Space: O(1)

Skeleton Code:

```java
public int maxArea(int[] height) {
    int l = 0, r = height.length - 1;
    int maxArea = 0;

    while (l < r) {
        int h = Math.min(height[l], height[r]);
        int width = r - l;
        int area = h * width;
        maxArea = Math.max(maxArea, area);

        // Move shorter pointer
        if (height[l] < height[r]) {
            l++;
        } else {
            r--;
        }
    }

    return maxArea;
}
```

Gotchas:

X Don't use leftMax/rightMax (that's Trapping Rain Water!)

OK Use height[l] and height[r] directly

! Water is BETWEEN lines, not on top

! Area = min(height[l], height[r]) × (r - l)

3.11 11. Trapping Rain Water

Problem: Calculate trapped rainwater on top of bars
Key Insight: Water level = min(leftMax, rightMax) - height
Complexity:

- Time: O(n)
- Space: O(1)

Skeleton Code:

```java
public int trap(int[] height) {
    if (height == null || height.length == 0) return 0;

    int l = 0, r = height.length - 1;
    int leftMax = height[l], rightMax = height[r];
    int res = 0;

    while (l < r) {
```

```
        if (leftMax < rightMax) {
            l++;
            leftMax = Math.max(leftMax, height[l]);
            res += leftMax - height[l];
        } else {
            r--;
            rightMax = Math.max(rightMax, height[r]);
            res += rightMax - height[r];
        }
    }

    return res;
}
```

Gotchas:

! Move pointer FIRST, then calculate at NEW position

! All in SAME iteration (not next iteration!)

! Water ON TOP of bars, not between them

OK Use leftMax and rightMax (running maximums)

Order of operations:

```
if (leftMax < rightMax) {
    l++;              // Step 1: Move FIRST
    leftMax = ...     // Step 2: Update at NEW position
    res += ...        // Step 3: Calculate at NEW position
}
All three steps happen in SAME iteration!
```

3.12 12. Find Minimum in Rotated Sorted Array

Problem: Find minimum in rotated sorted array

Key Insight: Binary search, compare mid with right

Complexity:

- Time: O(log n)
- Space: O(1)

Skeleton Code:

```
public int findMin(int[] nums) {
    int left = 0, right = nums.length - 1;

    // Early exit: not rotated
    if (nums[left] < nums[right]) return nums[left];

    while (left < right) {  // Note: <, not <=
        int mid = left + (right - left) / 2;

        if (nums[mid] > nums[right]) {
            left = mid + 1;  // Min in right half
        } else {
            right = mid;     // Min in left half (or at mid)
        }
    }

    return nums[left];
}
```

Gotchas:

OK Use `left < right` (finding position)

OK Use `right = mid` (keep mid in search)

! Compare with nums[right], not nums[left]

! Early exit: if nums[left] < nums[right], not rotated

Why compare with right?
nums[mid] > nums[right] → rotation point is in right half
nums[mid] < nums[right] → rotation point is in left half (or no rotation)

3.13 13. Search in Rotated Sorted Array

Problem: Search target in rotated sorted array
Key Insight: Determine which half is sorted, then check if target is there
Complexity:

- Time: O(log n)
- Space: O(1)

Skeleton Code:

```java
public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {  // Note: <=, not <
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) return mid;

        // Check which half is sorted
        if (nums[left] <= nums[mid]) {
            // Left half sorted
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            // Right half sorted
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }

    return -1;
}
```

Gotchas:

OK Use `left <= right` (finding exact value)

OK Use `right = mid - 1` and `left = mid + 1`

! Check: nums[left] <= nums[mid] (not <, handles duplicates at boundaries)

! Must check if target is in sorted half

3.14 14. Maximum Subarray (Kadane's Algorithm)

Problem: Find subarray with maximum sum

Key Insight: At each position, extend or start fresh

Complexity:

- Time: O(n)
- Space: O(1)

Skeleton Code:

```java
public int maxSubArray(int[] nums) {
    int currMax = nums[0];
    int maxSum = nums[0];

    for (int i = 1; i < nums.length; i++) {
        currMax = Math.max(nums[i], currMax + nums[i]);
        maxSum = Math.max(maxSum, currMax);
    }

    return maxSum;
}
```

Gotchas:

OK Initialize with nums[0] (handles single element)

OK Start loop from i = 1

! Decision: extend (currMax + nums[i]) or start fresh (nums[i])

! If cumulative sum is negative, it can't help future elements

Why it works:

At position i, we have two choices:

1. Extend previous subarray: currMax + nums[i]
2. Start new subarray: nums[i]

Choose whichever is larger! If previous sum is negative, it drags us down → start fresh.

4 Skeleton Code Templates

4.1 Template 1: Variable Sliding Window

```
public ReturnType slidingWindow(DataType[] arr, int condition) {
    Map<Type, Integer> window = new HashMap<>();
    int left = 0;
    ReturnType result = initialValue;

    for (int right = 0; right < arr.length; right++) {
        // Expand window
        window.put(arr[right], window.getOrDefault(arr[right], 0) + 1);

        // Shrink window if condition violated
        while (conditionViolated) {
            window.put(arr[left], window.get(arr[left]) - 1);
            if (window.get(arr[left]) == 0) {
                window.remove(arr[left]);
            }
            left++;
        }

        // Update result
        result = updateResult(result, right - left + 1);
    }

    return result;
}
```

4.2 Template 2: Fixed Sliding Window

```
public ReturnType fixedWindow(int[] arr, int k) {
    // Calculate first window
    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += arr[i];
    }

    ReturnType result = sum;

    // Slide window
    for (int i = k; i < arr.length; i++) {
        sum = sum - arr[i - k] + arr[i];
        result = updateResult(result, sum);
    }

    return result;
}
```

4.3 Template 3: Two Pointers (Opposite Ends)

```
public ReturnType twoPointers(int[] arr) {
    int left = 0, right = arr.length - 1;
    ReturnType result = initialValue;

    while (left < right) {
        // Calculate current value
```

```
        Type current = calculate(arr[left], arr[right]);
        result = updateResult(result, current);

        // Move pointers based on strategy
        if (shouldMoveLeft) {
            left++;
        } else {
            right--;
        }
    }

    return result;
}
```

4.4 Template 4: Binary Search (Find Position)

```
public int binarySearchPosition(int[] arr) {
    int left = 0, right = arr.length - 1;

    while (left < right) {  // Note: <
        int mid = left + (right - left) / 2;

        if (condition(arr[mid])) {
            left = mid + 1;
        } else {
            right = mid;  // Keep mid
        }
    }

    return left;  // or nums[left]
}
```

4.5 Template 5: Binary Search (Find Value)

```
public int binarySearchValue(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {  // Note: <=
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}
```

4.6 Template 6: HashMap Frequency Counting

```
public ReturnType frequencyCount(Type[] arr) {
    Map<Type, Integer> freq = new HashMap<>();

```

```
    // Count frequencies
    for (Type item : arr) {
        freq.put(item, freq.getOrDefault(item, 0) + 1);
    }

    // Process frequencies
    for (Map.Entry<Type, Integer> entry : freq.entrySet()) {
        Type key = entry.getKey();
        int count = entry.getValue();
        // Process...
    }

    return result;
}
```

5 Common Gotchas & Pitfalls

5.1 1. Integer Comparison in HashMap

```
// [X] WRONG
if (map.get(key) == map.get(otherKey))  // Compares references!

// [OK] CORRECT
if (map.get(key).equals(map.get(otherKey)))  // Compares values
```

5.2 2. Sliding Window - When to Update Result

```
// [X] WRONG - Update outside while loop
while (conditionViolated) {
    shrink();
}
result = update();  // Only updates when condition violated!

// [OK] CORRECT - Update inside for loop
for (int right = 0; right < n; right++) {
    expand();
    while (conditionViolated) {
        shrink();
    }
    result = update();  // Updates every iteration
}
```

5.3 3. Array Bounds with Pointers

```
// [X] WRONG
while (left < right) {
    if (condition) {
        left++;
        process(arr[left]);  // May be out of bounds!
    }
}

// [OK] CORRECT
while (left < right) {
    if (condition) {
        left++;
        if (left < arr.length) {  // Check bounds
            process(arr[left]);
        }
    }
}
```

5.4 4. Binary Search - Infinite Loop

```
// [X] WRONG - Can cause infinite loop
while (left < right) {
    int mid = (left + right) / 2;
    if (condition) {
        left = mid;  // If left = mid = 0, right = 1, infinite loop!
    } else {
        right = mid - 1;
```

```
    }
}

// [OK] CORRECT - Always make progress
while (left < right) {
    int mid = left + (right - left) / 2;
    if (condition) {
        left = mid + 1;  // Always moves
    } else {
        right = mid;
    }
}
```

5.5 5. Substring End Index

```
// [X] WRONG
String result = s.substring(start, end);  // Excludes character at 'end'

// [OK] CORRECT
String result = s.substring(start, end + 1);  // Includes character at 'end'
```

5.6 6. Early Return in Loops

```
// [X] WRONG - Modifies list while iterating
for (int i = 0; i < list.size(); i++) {
    if (condition) {
        list.remove(i);  // Size changes, indices shift!
    }
}

// [OK] CORRECT - Iterate backwards
for (int i = list.size() - 1; i >= 0; i--) {
    if (condition) {
        list.remove(i);
    }
}
```

5.7 7. Stack Empty Check

```
// [X] WRONG
if (stack.pop() != expected) {  // Crashes if empty!
    return false;
}

// [OK] CORRECT
if (stack.isEmpty() || stack.pop() != expected) {
    return false;
}
```

5.8 8. Max/Min Initialization

```
// [X] WRONG - Fails for all-negative arrays
int maxSum = 0;

// [OK] CORRECT
```

```
int maxSum = Integer.MIN_VALUE;
// OR better: initialize with first element
int maxSum = arr[0];
```

6 Your Knowledge Gaps & Areas to Review

6.1 1. Confusing Similar Problems

Issue: Container With Most Water vs Trapping Rain Water

What you mixed up:

Container With Most Water:

- Use: height[l], height[r] directly
- Water: BETWEEN two lines
- Move: Pointer of shorter height

Trapping Rain Water:

- Use: leftMax, rightMax (running maximums)
- Water: ON TOP of bars
- Move: Pointer of smaller max

How to remember:

- Container = 2 walls holding water between them
- Trapping = Multiple bars trapping water on top

6.2 2. Order of Operations in Same Iteration

Confusion: Does move-then-calculate happen in same or next iteration?

Answer: ALL in SAME iteration!

```
while (l < r) {
    if (leftMax < rightMax) {
        l++;                        // <- Step 1
        leftMax = Math.max(...) // <- Step 2  } All in
        res += leftMax - ...    // <- Step 3  } SAME iteration!
    }
}
```

Key insight: We NEVER calculate at initial boundary position (0 or n-1)

6.3 3. Binary Search Loop Conditions

Confusion: When to use < vs <=?

Rule:

left < right:

- Finding POSITION
- Use: right = mid
- Return: nums[left]

left <= right:

- Finding EXACT VALUE
- Use: right = mid - 1
- Return: index or -1

Memory aid: If you see `mid - 1` or `mid + 1` for BOTH branches, use `<=`

6.4 4. Math.max() in Sliding Window

Confusion: Why Math.max() when updating left pointer?

```
left = Math.max(left, lastSeen.get(c) + 1);
```

Why needed?
lastSeen[c] might point to position BEFORE current left
Example: "abba"
When right = 3 (second 'a'):

- lastSeen['a'] = 0
- But left is already at 2
- Can't move backwards!
- Math.max ensures left only moves forward

6.5 5. When HashMap vs Array

Confusion: Always optimize to array?

Decision tree:
Is character set limited and known? (ASCII/lowercase)

- YES → Use array (O(1) space, faster)
- NO → Use HashMap (flexible)

Need to support Unicode/arbitrary characters?

- YES → Must use HashMap
- NO → Can use array

7 Quick Reference Tables

7.1 Complexity Cheat Sheet

Problem	Time	Space	Pattern
Product Except Self	O(n)	O(1)	Prefix/Suffix
Subarray Sum = K	O(n)	O(n)	Prefix Sum + HashMap
Top K Frequent	O(n)	O(n)	Bucket Sort
Valid Parentheses	O(n)	O(n)	Stack
Group Anagrams	O(n×k)	O(n×k)	HashMap
Longest Substring	O(n)	O(min(n,m))	Sliding Window
Min Window Substring	O(n+m)	O(k)	Sliding Window
Character Replacement	O(n)	O(1)	Sliding Window
Max Average	O(n)	O(1)	Fixed Window
Container Water	O(n)	O(1)	Two Pointers
Trap Rain Water	O(n)	O(1)	Two Pointers
Find Minimum Rotated	O(log n)	O(1)	Binary Search
Search Rotated	O(log n)	O(1)	Binary Search
Max Subarray	O(n)	O(1)	Kadane's

7.2 When to Use Which Data Structure

Need	Use	Example Problem
Matching pairs	Stack	Valid Parentheses
Frequency counting	HashMap	Top K Frequent
Last seen index	HashMap	Longest Substring
Prefix sums	HashMap	Subarray Sum = K
Character frequency	Array[26/128]	Min Window, Character Replacement
Order matters	ArrayList	Top K (buckets)

7.3 Optimization Checklist

Can you...

- ☐ Use array instead of HashMap? (limited charset)
- ☐ Use sliding window instead of nested loops? (contiguous)
- ☐ Use two pointers instead of extra space? (sorted or opposite ends)
- ☐ Use prefix sum to avoid recalculation?
- ☐ Use constant space instead of auxiliary array?
- ☐ Avoid sorting if bucket sort works?
- ☐ Track running max/min instead of recalculating?

8 Tips & Tricks

8.1 1. Recognize the Pattern

Keywords → Pattern

Keyword	Pattern
"Contiguous subarray/substring"	Sliding Window
"All pairs" or "two elements"	Two Pointers
"Sorted or rotated sorted"	Binary Search
"Matching/balanced"	Stack
"Frequency/count"	HashMap
"Top K / K largest/smallest"	Heap or Bucket Sort
"Maximum/Minimum subarray sum"	Kadane's or DP

8.2 2. Ask These Questions

Before coding:

1. What's the constraint? (contiguous? sorted? unique?)
2. What's the brute force? (helps understand problem)
3. Can I use two pointers? (opposite ends or same direction?)
4. Can I use sliding window? (contiguous + optimization)
5. Do I need to track frequency? (HashMap or Array?)
6. Is there a greedy choice? (local optimum → global)

8.3 3. Debugging Checklist

When your code fails:

☐ Did you initialize variables correctly?

☐ Are you checking bounds before accessing arrays?

☐ Did you handle empty input?

☐ Did you handle single element?

☐ Are you using .equals() for Integer comparison?

☐ Did you check stack.isEmpty() before pop()?

☐ Are you using correct loop condition (< vs <=)?

☐ Did you update ALL related variables when moving pointers?

☐ Are you calculating indices correctly (off-by-one)?

☐ Did you handle negative numbers?

8.4 4. Optimization Strategies

$O(n^2) \rightarrow O(n)$:

- Use HashMap to track previous values
- Use sliding window for contiguous
- Use two pointers if sorted

$O(n \log n) \rightarrow O(n)$:

- Avoid sorting if possible
- Use bucket sort if value range is limited
- Use linear scan with HashMap

$O(n)$ space $\rightarrow O(1)$:

- Reuse input array
- Use constant-size array instead of HashMap
- Use two pointers instead of auxiliary space

8.5 5. Edge Cases to Always Test

☐ Empty array/string: []

☐ Single element: [5]

☐ Two elements: [1, 2]

☐ All same: [5, 5, 5]

☐ All negative: [-1, -2, -3]

☐ All positive: [1, 2, 3]

☐ Mixed: [-1, 2, -3]

☐ Minimum/Maximum values: [Integer.MIN_VALUE]

☐ Already sorted: [1, 2, 3]

☐ Reverse sorted: [3, 2, 1]

☐ Duplicates: [1, 1, 2, 2]